

INF3201- Parallel Programming

Assignment 2 - Raster Image Filtering

Jens Christian Valen Leynse
UiT
Tromsø, Norway
jeley4465@uit.no

Sivert Bottolfsen
UiT
Tromsø, Norway
sbo101@uit.no

1 Introduction

The project aims to implement and optimize raster image filtering techniques using both sequential and parallel approaches. The goal is to enhance image processing performance by utilizing parallel computing capabilities through MPI and OpenMP. The two primary filters employed in this project are the Box Blur and Sobel filters, which are widely used for image enhancement and feature detection, respectively.

2 Sequential Solution

The sequential image filtering solution applies the Box Blur and Sobel filters to a given image in a straightforward manner. This implementation systematically processes each pixel and its neighbors to compute the new pixel values based on the defined filters.

2.1 Initialization and Execution

The main function initializes the program by loading the input BMP image, defining the output image, and allocating memory for the filtered images. It then applies the desired filters by iterating through each pixel, calculating the new pixel values using the corresponding filter's convolution matrices, and storing the results.

2.2 Box Blur Implementation:

The Box Blur filter works by averaging the pixel values in a defined neighborhood around each pixel. The algorithm iterates through each pixel of the

image, calculates the average of surrounding pixel values using a 3x3 kernel, and assigns this average to the output image. This results in a blurred effect, which reduces noise and detail.

2.3 Sobel Filter Implementation

The Sobel filter, used for edge detection, first converts the RGB image to grayscale using the luminosity method, defined as $\text{grayscale} = 0.3R + 0.59G + 0.11B$. Next, it applies the Sobel kernels (G_x and G_y) to compute the gradient magnitude at each pixel. The final pixel value is determined by combining the results of the convolution with both kernels.

2.4 Running the Sequential Solution

To run the sequential solution, follow these steps:

- Compile the Code:

```
make run
```

3 Parallel Design Solution

The parallel design solution employs the Message Passing Interface (MPI) combined with OpenMP to enhance the performance of raster image filtering by distributing the workload across multiple processes and threads.

3.1 Parallelization Strategy

The parallelization utilizes a master-worker architecture, where the master process coordinates the workload distribution among the worker processes, each of which applies the Box Blur and Sobel filters to specific image tiles.

- Workload Distribution:
The original image is divided into tiles, with each worker process assigned at least one tile to process. The size of each tile is determined based on the number of processes used.
- Filter Application:
Each worker process applies the specified filter to its assigned tile. For the Sobel filter, workers first convert their tile to grayscale and then apply the Sobel kernels.

- **Communication:**
Processes communicate to exchange pixel data at the edges of their tiles, ensuring that the filters can correctly process pixels that require neighboring pixel information from adjacent tiles.
- **OpenMP Integration:**
Inside each MPI process, OpenMP is used to parallelize the computations performed on each tile. This involves parallelizing the loops that apply the filters to improve the efficiency of pixel processing.

3.2 Running the Parallel Implementation

To execute the parallel image filtering program, follow these steps:

- **Execution:**
Run the compiled program using the command:


```
make run N="number of processes"
```
- This command specifies the number of processes to use for the filtering operation.

4 Time Performance Analysis

Experiments are to be conducted to evaluate the performance of both the sequential and parallel implementations of the image filtering algorithms. The performance metrics include execution time and scalability based on the image size and the number of processes used.

- **Execution Time Measurements:**
Sequential execution times are recorded for varying image sizes.
Parallel execution times are measured for different numbers of processes, analyzing the performance improvements gained through parallelization.
- **Results:**
Will be added when time measurements can be taken.

5 Discussion

Positive Aspects

- **Parallel Solution:**
The parallel implementation should ideally reduce execution time as more processes are utilized. But for the current implementation it is a bit slower than the serial one.
- **Running on the cluster:**
While the program runs locally, when we tried running on the cluster, we got some errors. We would have addressed these errors, if we had more time.
- **Number of processes:**
The number of processes is currently set to 6. This is hardcoded as we have not implemented a optimal dynamic splitting of tiles.
- **Pixel lines**
We had some issues with lines in the output image to begin with. But by implementing communication between the processes tiles, we were able to remove this artifact, resulting in the desired image output, without tile border lines.
- **use of openMP:**
The use of openMP is not very fast in its current state. This is likely because the threads often are blocked by a critical section. If given more time, we would have separated rendering of pixels that required MPI communication into its own thread, while the others could render the parts of the image that does not need pixels from other processes. This way, no threads would have to wait for the critical section of other threads.
- **Scalability:**
The solution would not scale very well in its current state, as a increase of processes could lead to issues with pixel sharing between the processes.

Negative Aspects

- **Parallel Solution:**
Sequential Solution:

The sequential implementation suffers from longer execution times and is impractical for processing large images.

- Parallel Solution:
Overhead from inter-process communication can affect performance, particularly when dealing with smaller images or fewer processes.⁶

6 Summary

The project should illustrate the advantages of parallel processing in optimizing image filtering techniques. The parallel version should reduce the execution time compared to the sequential approach, especially for larger input sizes. However, performance gains can be constrained by communication overhead.